At the beginning of a software project, the business need is often expressed informally as part of a simple conversation. The conversation presented in the sidebar is typical.

SAFEHOME[13]



How a Project Starts

The scene: Meeting room at CPI Corporation, a (fictional) company that makes consumer products for home and commercial use.

The players: Mal Golden, senior manager, product development; Lisa Perez, marketing manager; Lee Warren, engineering manager; Joe Camalleri, executive vice president, business development

The conversation:

Joe: Okay, Lee, what's this I hear about your folks developing a what? A generic universal wireless box?

Lee: It's pretty cool . . . about the size of a small matchbook . . . we can attach it to sensors of all kinds, a digital camera, just about anything. Using the 802.11n wireless protocol. It allows us to access the device's output without wires. We think it'll lead to a whole new generation of products.

Joe: You agree, Mal?

Mal: I do. In fact, with sales as flat as they've been this year, we need something new. Lisa and I have been doing a little market research, and we think we've got a line of products that could be big.

Joe: How big . . . bottom line big?

Mal (avoiding a direct commitment): Tell him about our idea, Lisa.

Lisa: It's a whole new generation of what we call "home management products." We call 'em *SafeHome*. They use the new wireless interface, provide homeowners or small-businesspeople with a system that's controlled by their PC—home security, home surveillance, appliance and device control—you know, turn down the home air conditioner while you're driving home, that sort of thing.

Lee (jumping in): Engineering's done a technical feasibility study of this idea, Joe. It's doable at low manufacturing cost. Most hardware is off the shelf. Software is an issue, but it's nothing that we can't do.

Joe: Interesting. Now, I asked about the bottom line.

Mal: PCs and tablets have penetrated over 70 percent of all households in the USA. If we could price this thing right, it could be a killer app. Nobody else has our wireless box . . . it's proprietary. We'll have a 2-year jump on the competition. Revenue? Maybe as much as $30 to $40 million in the second year.

Joe (smiling): Let's take this to the next level. I'm interested.

With the exception of a passing reference, software was hardly mentioned as part of the conversation. And yet, software will make or break the *SafeHome* product line. The engineering effort will succeed only if *SafeHome* software succeeds. The market will accept the product only if the software embedded within it properly meets the customer's (as yet unstated) needs. We'll follow the progression of *SafeHome* software engineering in many of the chapters that follow.

13 *SafeHome* will be used throughout this book to illustrate the inner workings of project teams as they build a software product. The company, the project, and the people are purely fictitious, but the situations and problems are real.

1.6 SUMMARY

Software is the key element in the evolution of computer-based systems and products and one of the most important technologies on the world stage. Over the past 60 years, software has evolved from a specialized problem-solving and information analysis tool to an industry in itself. Yet we still have trouble developing high-quality software on time and within budget.

Software—programs, data, and descriptive information—addresses a wide array of technology and application areas. Legacy software continues to present special challenges to those who must maintain it.

Software engineering encompasses process, methods, and tools that enable complex computer-based systems to be built in a timely manner with quality. The software process incorporates five framework activities—communication, planning, modeling, construction, and deployment—that are applicable to all software projects. Software engineering practice is a problem-solving activity that follows a set of core principles. As you learn more about software engineering, you'll begin to understand why these principles should be considered when beginning any software project.

PROBLEMS AND POINTS TO PONDER

1.1. Provide at least five additional examples of how the law of unintended consequences applies to computer software.

1.2. Provide a number of examples (both positive and negative) that indicate the impact of software on our society.

1.3. Develop your own answers to the five questions asked at the beginning of Section 1.1. Discuss them with your fellow students.

1.4. Many modern applications change frequently—before they are presented to the end user and then after the first version has been put into use. Suggest a few ways to build software to stop deterioration due to change.

1.5. Consider the seven software categories presented in Section 1.1.2. Do you think that the same approach to software engineering can be applied for each? Explain your answer.

1.6. As software becomes more pervasive, risks to the public (due to faulty programs) become an increasingly significant concern. Develop a doomsday but realistic scenario in which the failure of a computer program could do great harm, either economic or human.

1.7. Describe a process framework in your own words. When we say that framework activities are applicable to all projects, does this mean that the same work tasks are applied for all projects, regardless of size and complexity? Explain.

1.8. Umbrella activities occur throughout the software process. Do you think they are applied evenly across the process, or are some concentrated in one or more framework activities?

Design element: Quick Look icon magnifying glass: © Roger Pressman

PART

One

THE SOFTWARE PROCESS

In this part of *Software Engineering: A Practitioner's Approach,* you'll learn about the process that provides a framework for software engineering practice. These questions are addressed in the chapters that follow:

- What is a software process?
- What are the generic framework activities that are present in every software process?
- How are processes modeled, and what are process patterns?
- What are the prescriptive process models, and what are their strengths and weaknesses?
- Why is *agility* a watchword in modern software engineering work?
- What is agile software development, and how does it differ from more traditional process models?

Once these questions are answered, you'll be better prepared to understand the context in which software engineering practice is applied.

CHAPTER

2 PROCESS MODELS

Building computer software is an iterative social learning process, and the outcome, something that Baetjer [Bae98] would call "software capital," is an embodiment of knowledge collected, distilled, and organized as the process is conducted.

But what exactly is a software process from a technical point of view? Within the context of this book, we define a *software process* as a framework for the activities, actions, and tasks required to build high-quality software. Is "process" synonymous with "software engineering"? The answer is yes and no. A software process defines the approach that is taken as software is engineered. But software engineering also encompasses technologies that populate the process—technical methods and automated tools.

More important, software engineering is performed by creative, knowledgeable people who should adapt a mature software process so that it is appropriate for the products that they build and the demands of their marketplace.

KEY CONCEPTS

evolutionary process model 29
generic process model 21
process assessment 24
process flow 24
process improvement 24
process patterns 24
prototyping 26
spiral model 29
task set 21
Unified Process 31
waterfall model 25



QUICK LOOK

What is it? When you work to build a product or system, it's important to follow a series of predictable steps (a road map) that helps you deliver a high-quality product on time. This road map is called a "software process."

Who does it? Software engineers adapt a process to their needs and then follow it. The people who have requested the software also have a role to play in the process of defining, building, and testing it.

Why is it important? A process provides stability, control, and organization to an activity so that it does not become chaotic. However, a modern software engineering process must be "agile." It must include only those activities, controls, and work products that are appropriate for the project team and the product that is to be produced.

What are the steps? The process that you adopt depends on the software that you're building. A process might be appropriate for creating software for an aircraft avionics system but may not work well for the creation of a mobile app or video game.

What is the work product? The work products are the programs, documents, and data produced by the engineering activities and tasks included in the process.

How do I ensure that I've done it right? The quality, timeliness, and long-term viability of the product built are the best indicators of the success of the process used.

2.1 A Generic Process Model

In Chapter 1, a process was defined as a collection of activities, actions, and tasks that are performed when some work product is to be created. Each of these activities, actions, and tasks resides within a framework or model that defines their relationship with the process and with one another.

The software process is represented schematically in Figure 2.1. Referring to the figure, each framework activity is populated by a set of software engineering actions. Each software engineering action is defined by a *task set* that identifies the work tasks that are to be completed, the work products that will be produced, the quality assurance points that will be required, and the milestones that will be used to indicate progress.

As we discussed in Chapter 1, a generic process framework for software engineering defines five framework activities—**communication, planning, modeling, construction,** and **deployment.** In addition, a set of umbrella activities—project tracking and control,

FIGURE 2.1

A software process framework

